UMLytics – AI-Powered UML Generation and Design Intelligence Tool

Project Team

Ammar Bin Omer	24I-0500
Haris Zahid	24I-0643
Shahmeer Jadoon	24I-0879

1. Project Overview:

UMLytics is a desktop application that assists developers in creating, analysing, and refining UML class diagrams with AI support. The system provides a chat-based interface for conversational UML creation and clarification, automatic class diagram generation from natural language descriptions or parsed source code, and AI driven design critique grounded in Software Design and Architecture principles. Users can interactively edit diagrams (e.g., classes, attributes, and relationships), request AI assisted class skeletons or structural suggestions for early stage design, and perform high level analysis of uploaded UML diagram images. Optional voice input is provided as an accessibility feature. The application is implemented as a Java based desktop client with a relational database for managing projects, diagrams, chat history, and evaluation results.

2. Motivation:

Effective software design depends on clear modelling and informed decision-making based on principles such as coupling, cohesion, and the SOLID guidelines. In the Software Design and Architecture course, students often struggle to translate theoretical concepts into concrete diagrams and to iteratively improve designs before implementation. Existing UML tools primarily focus on manual diagram creation or reverse engineering, offering limited support for principle-based critique and learning oriented refinement. UMLytics is motivated by the need for an intelligent, interactive tool that supports early stage design exploration and helps users understand architectural trade offs without relying on full round-trip code generation or research-level automation.

3. Objectives:

- Enable conversational UML modelling through a chat-based interface that supports natural-language design descriptions and iterative clarification.
- Automatically generate UML class diagrams from user descriptions and from parsed object-oriented source code.
- Evaluate software design quality using SDA principles such as coupling, cohesion, and SOLID, and present structured feedback.
- Support interactive editing of UML class diagrams to refine designs before or after AI feedback.
- Provide AI-assisted class skeletons or code-structure suggestions to support early-stage

design and learning.
- Offer high level UML diagram image interpretation and optional voice input as accessibility features.
- Persist project data, diagrams, and evaluation history using a relational database.

4. Expected Outcomes:

The project will result in a functional Java-based desktop application that integrates chat driven UML creation, automatic diagram generation, interactive editing, and AI-assisted design critique. The system will include a well structured relational database, clear documentation of application architecture, and explicit mapping of SDA concepts within both the AI critique process and the system's own design. Sample use cases and scenarios will demonstrate how UMLytics supports early-stage design understanding and improvement.

5. Project Scope:

In scope are: conversational UML creation; UML class diagram generation from natural-language input and parsed Java source code; AI based design critique using coupling, cohesion, and SOLID principles; interactive editing of classes, attributes, and relationships; AI assisted generation of class skeletons or structural suggestions for early-stage design; high level analysis of uploaded UML diagram images; optional voice input for accessibility; a Java Swing or JavaFX desktop interface; and relational database support using SQL or Oracle.

Out of scope are: full automatic round-trip code generation or bidirectional code–diagram synchronisation; support for diagram types beyond UML class diagrams; real-time collaboration; web or mobile deployment; and formal research or empirical evaluation. AI-generated outputs are framed as educational and early-design support rather than production-ready solutions.

6. Tools and Technologies:

- **Frontend:** Java with Java Swing the desktop graphical user interface.
- **Backend and persistence:** Relational database using SQL accessed through JDBC or a lightweight persistence layer.
- **Language parsing and diagram modelling:** Java based parsing libraries and in-memory UML models for diagram generation, editing, and export.
- **AI integration:** An AI/LLM API (institution approved) for chat based UML generation, design critique, structural suggestions, and high-level image interpretation, with prompts aligned to SDA principles.
- **Optional features:** Speech-to-text support for voice input and standard image-processing libraries for UML diagram uploads.
- **Development tools:** IDE IntelliJ IDEA, build tools such as Maven or Gradle, and Git for version control.